Human-Like Bots for Tactical Shooters Using Compute-Efficient Sensors

Niels Justesen, Maria Kaselimi, Sam Snodgrass, Miruna Vozaru, Matthew Schlegel, Jonas Wingren, Gabriella A. B. Barros, Tobias Mahlmann, Shyam Sudhakaran, Wesley Kerr, Albert Wang, Christoffer Holmgård, Georgios N. Yannakakis, Sebastian Risi, and Julian Togelius

***Abstract*—Artificial intelligence (AI) has enabled agents to master complex video games, from first-person shooters, such as *Counter-Strike*, to real-time strategy games, such as *StarCraft II*, and racing games such as *Gran Turismo*. While these achievements are notable, applying these AI methods in commercial video game production remains challenging due to computational constraints. In commercial scenarios, the majority of computational resources are allocated to 3-D rendering, leaving limited capacity for AI methods, which often demand high computational power, particularly those relying on pixel-based sensors. Moreover, the gaming industry prioritizes creating human-like behavior in AI agents to enhance player experience, unlike academic models that focus on maximizing game performance. This article introduces a novel methodology for training neural networks via imitation learning to play a complex, commercial-standard, VALORANT-like 2v2 tactical shooter game, requiring only modest CPU hardware during inference. Our approach leverages an innovative, pixel-free perception architecture using a small set of ray-cast sensors, which capture essential spatial information efficiently. These sensors allow AI to perform competently without the computational overhead of traditional methods. Models are trained to mimic human behavior using supervised learning on human trajectory data, resulting in realistic and engaging AI agents. Human evaluation tests confirm that our AI agents provide human-like gameplay experiences while operating efficiently under computational constraints. This offers a significant advancement in AI model development for tactical shooter games and possibly other genres.**

***Index Terms*—Games, human behavior, human evaluation, imitation learning.**

Received 8 December 2024; revised 17 May 2025 and 26 June 2025; accepted 22 July 2025. Date of publication 30 July 2025; date of current version 17 December 2025. Recommended by Associate Editor Kyung-Joong Kim. *(Corresponding author: Niels Justesen.)*

This work involved human subjects or animals in its research. Approval of all ethical and experimental procedures and protocols was granted by the Institutional Review Board (IRB) of the University of Malta and performed in line with the Declaration of Helsinki.

Niels Justesen, Maria Kaselimi, Sam Snodgrass, Miruna Vozaru, Matthew Schlegel, Jonas Wingren, Gabriella A. B. Barros, Tobias Mahlmann, Shyam Sudhakaran, Christoffer Holmgård, Georgios N. Yannakakis, Sebastian Risi, and Julian Togelius are with modl.ai, Copenhagen 2200, Denmark (e-mail: niels@modl.ai; maria@modl.ai; sam@modl.ai; miruna@modl.ai; matthew@modl.ai; jonas.wingren@modl.ai; gabriella@modl.ai; tobias@modl.ai; shyam@modl.ai; christoffer@modl.ai; georgios@modl.ai; sebastian@modl.ai; julian@modl.ai).

Wesley Kerr and Albert Wang are with Riot Games, Los Angeles, CA 90064 USA (e-mail: wkerr@riotgames.com; alwang@riotgames.co).

This article has supplementary downloadable material available at https://doi.org/10.1109/TG.2025.3593919, provided by the authors.

Digital Object Identifier 10.1109/TG.2025.3593919

I. Introduction

TRANSITIONING gameplay agents from academia to industry present several challenges. Advanced artificial intelligence (AI) models often require significant computational resources, making them impractical for consumer-grade hardware. In addition, academic models typically aim for optimal performance [1], whereas the industry prioritizes human-like behavior to improve the player experience [2], necessitating a different approach for defining reward functions. There is also the challenge of integrating AI into complex game environments where most computational resources are dedicated to tasks, such as 3-D rendering, leaving limited capacity for AI processes.

The training process of the AI algorithms often requires processing of large datasets, especially when using pixel-based inputs that contain vast amounts of visual information [3]. In addition, the models themselves are typically large and complex, consisting of many layers and parameters, which require significant computational power and memory to train and run [3], [4]. The need for real-time performance in video games further complicates matters, as AI agents must make decisions and react almost instantaneously, a task that is resource-intensive and difficult to achieve with the limited computational budget available in consumer hardware. These challenges highlight the need for more efficient algorithms and innovative approaches to make AI agents practical and effective in commercial video games.

The believability [5] of AI agents is crucial for creating immersive and engaging experiences in multiplayer games, highlighting another key difference between academic research and industry applications. AI agents that can play multiplayer games well and believably are useful in numerous ways [6]. They can act as opponents or teammates when suitable human players are unavailable, serve as stand-ins for specific players who drop out, and even emulate a particular player's style for others to play against. In addition, they aid game designers by automatically playtesting maps, items, and game tweaks, and they can be integral to tutorials. In scenarios where human demonstrations are available, it is intuitively preferable for an agent to learn directly from these demonstrations, thereby fostering more human-like gameplay.

Here, we introduce a method for training compute-efficient deep neural networks to play a multiplayer team-based first-person shooter in a human-like fashion via imitation learning. For models to be successfully deployed in the production of

modern 3-D video games, the inference process within a frame must be completed in single-digit milliseconds on a CPU thread. Therefore, we train relatively small networks and use simulated sensors rather than pixels as inputs. We evaluate our trained bots for performance, inference time, and believability. Performance evaluation is done through match-ups between bots in the game, as well as by playing human versus bot matches. Inference time is validated by comparing the time required by our models with that of existing models designed for games, such as *Counter-Strike: Global Offensive (CS:GO)*. Believability is evaluated using Turing-test-like experiment protocol on video recordings.

The contributions of this article are summarized as follows.

1) *Multisensor Data Integration in Virtual Gameplay Environments:* Our models utilize sensors to capture spatial characteristics in gamer's movements within virtual environments without delving into pixel-based processing to optimize bot's movements, ensuring that they navigate virtual environments seamlessly and react appropriately to dynamic changes. The data gathered from these sensors are used for training machine learning (ML) bots, enabling compute-efficient models that can be deployed in commercial video games.
2) *Practicality in AI Bots for Gameplay:* The proposed models provide accurate predictions with minimal computational resources and low latency ensuring reliability and transparency. Prioritizing efficiency in inference allows game industry to deliver timely and dependable insights while reducing the risks of prolonged processing and high resource use. A supervised ML approach for temporal human trajectories is adopted to train models to closely mimic human behavior by learning directly from human trajectory data. This leads to realistic agent behaviors.
3) *Bot Human-Likeness Multifaceted Assessment:* In our comprehensive evaluation process, we assess the bots' human-likeness through three key steps. First, we analyze the similarity between distributions of bot behavior and generated data. Next, we examine spatial similarity using heatmaps to visualize bot movements in the virtual environment. Finally, we incorporate human evaluation through a questionnaire, gathering subjective feedback on the perceived beliavability of bot actions. Through this multifaceted approach, we strive to ensure that our bots exhibit behavior that is both realistic and engaging, enhancing the overall experience.

II. Background

In this section, we cover related works in imitation learning (see Section II-A) and its challenges when it is tasked to learn to play like a human (see Section II-B) in a multiplayer setting (see Section II-C). We end this background section with a discussion on agent believability and human-likeness (see Section II-D).

A. Imitation Learning

Imitation learning agents are normally trained to perform tasks from human demonstrations by learning a mapping between observations and actions [7]. The idea of learning by imitation has a longstanding history, but the field has been gaining increased interest in recent years [8] due to the growing number of available datasets. This surge of attention can be attributed to advancements in computing and sensing technologies, coupled with a growing demand for intelligent applications within video game environments [2]. Imitation learning in video games, however, faces several challenges, which can impact the effectiveness and robustness of the learned policies [7]. For instance, issues related to model stability and the limited exploration [9] present in the demonstrated behaviors of experts are some of the challenges that modern imitation learning algorithms face in that domain. We detail such challenges in the section that comes next (see Section II-B).

B. Learning to Play Like a Human: Core Challenges

Learning to play a game via imitation learning algorithms is challenging for a number of reasons. First of all, imitation learning algorithms face the risk of forgetting previously learned behaviors [10], especially in large imbalanced datasets or datasets that do not adequately represent the possible scenarios. The adoption of sequential modeling approaches and the enhancement of the architectures with long short-term memory (LSTM) layers prevents the models from catastrophically forgetting [11]. A recent example is the work of Pearce and Zhu [3] that applied LSTM-based behavioral cloning architectures to train playing bots for *CS:GO*. Recent studies adopt transformer-based architectures to handle long-range dependencies in sequences. Shafiullah et al. [12], for instance, used a behavior transformer to predict multimodal continuous actions while Dasari and Gupta [13] employed transformer architectures to one-shot imitation tasks. Similarly, in [14], transformer-based sequence models are leveraged as multitask multiembodiment policies across a wide range of video game environments, showcasing impressive results in few-shot out-of-distribution task learning. The study in [15] explores the use of a causal transformer to model human-like behavior in third-person shooter games, providing insights into how AI agents can be trained to replicate realistic player actions. Opposed to the current trend of employing large-scale models for imitating sequential decision-making tasks, such as game playing, in this article, we rely on simple yet efficient methods that exploit their rich sensor-based perception to act like human players. Our goal is to make such AI models operational and deployable to an actual commercial-standard game.

Imitation learning tends to exploit demonstrated behaviors, which may limit exploration and hinder the discovery of novel strategies or responses to unforeseen situations yielding limited generalization ability of the trained agent [16], [17]. As a response to this challenge, deep reinforcement learning (deep RL) has led to impressive recent AI breakthroughs with regards to agent generalizability in games, such as *Atari* [18], *Go* [4], *StarCraft* [6], *Dota* [19], *Gran Turismo* [20], and *Arena Breakout* [21]. In this work, we do not rely on the RL paradigm to learn to play a game better than any other human but instead attempt to design human-like bots in a tactical shooter game using a simple yet generic methodology. While the AI agents we train cannot necessarily transfer to other shooter games, their perception mechanism and training methods can. For the completeness of

the study, it is important to mention neuroevolution [5], [22], a promising approach in which evolutionary algorithms are employed to optimize neural network architectures and explore novel strategies. However, in our case, the availability of a rich dataset comprising human behavior makes imitation learning the most suitable approach for training agents to replicate human-like actions within the context of the tactical shooter game.

C. Learning to Play in Multiplayer Games

Game states in commercial-standard 3-D video games are often large and complex, defined by several variables—such as imagery data [3] and sensory data related to player positions, velocities, health, ammunition, etc. Such a game state space is often associated to a multidimensional and complex action space [23]. Arguably, the aforementioned spatiotemporal complexity of video game environments combined with a multiplayer setting poses collectively a significant challenge for AI algorithms to effectively explore and learn to play those games well. To address this challenge, a popular approach is to sample multiple actions individually to reduce the complexity of the action space at the cost of losing conditional dependencies [24].

Agents have been trained to play *Quake III Arena* (Activision, 1999) in *Capture the Flag* mode—i.e., where two multiplayer teams compete in capturing the flags of the opposing team [25]. Here, agents were trained by playing thousands of games, gradually learning successful strategies and competing against humans, even when their reaction times were slowed to match those of humans. To achieve strong agents in *StarCraft II*, deep RL has been combined with a multiagent tournament setup to produce agents with diverse strategies [6].

In contrary to the aforementioned studies, in this article, we focus on simple yet high-performing, compute-efficient, and deployable approaches for playing multiplayer video games in a human-like way.

D. Believability and Human-Like Agents

Crafting game-playing agents within a multiplayer setting that can effectively emulate human behavior——i.e., in a human-like fashion—adds a nonnegligible layer of complexity for imitation learning methods given the rich, diverse, and subjective nature of human play [26]. Various studies in the video game literature have focused on the creation and evaluation of bots in terms of believability and human-likeness (see [27], [28], and [29] among many). Specifically, in an early attempt, Ortega et al. [30] presented a method to assess the behavioral similarity of different agents playing *Super Mario Bros* to humans (or other agents), while Camilleri et al. [31] introduced a level generation method that maximizes the perceived believability of a *Super Mario Bros* player (human or not). A particularly notable effort in this space was the BotPrize competition [28], a multiyear initiative centered on developing agents that could mimic human behavior convincingly within the FPS game Unreal Tournament 2004 [32]. BotPrize holistically evaluated bots, in combat, navigation, timing, and decision-making, with the explicit goal of achieving human indistinguishability [22]. As such, it stands as one of the earliest and most comprehensive attempts to address human-like behavior in 3-D multiplayer shooter environments. Recently, Milani et al. [26] and Zuniga et al. [33] leveraged variants of Turing tests for video games and crowdsourced a dataset of free-form responses to gain insight into the navigation behaviors that human judges perceive as characteristic of AI versus humans. In this article, we tackle human-likeness in a far more holistic manner than mere navigation. We view human-like assessment as a multifactorial challenge and introduce both spatiotemporal metrics and multifaceted video game Turing-like tests to assess behavioral characteristics of tactical shooter play as a whole.

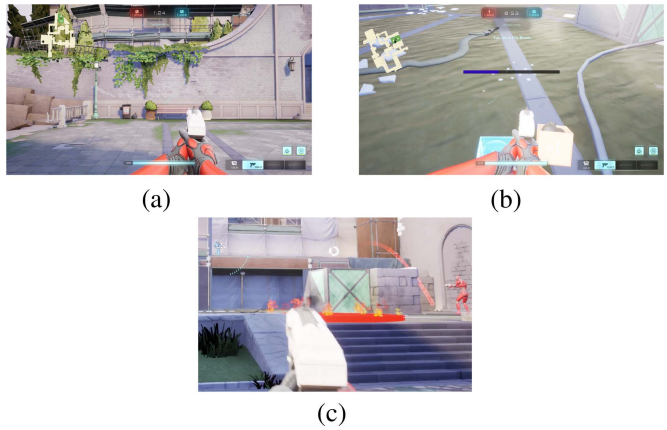
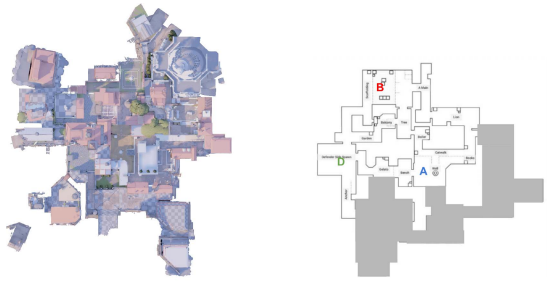


Fig. 1. Selected screenshots of the *Lyra:Ascent* game with (a) defending player at the start point, (b) a player planting the bomb, and (c) an area affected by a grenade.

III. *Lyra:Ascent* Game

For this study, we developed a 3-D tactical shooter game heavily inspired by *VALORANT* (Riot Games, 2020). We call this game *Lyra:Ascent*, and it is built using the Unreal Engine on top of the Lyra framework to ensure our test-bed follows commercial standards. The Lyra framework is Epic Games' official starter game for Unreal Engine, designed to showcase best practices in modern game development. Our game comes with a level similar to *Ascent* in VALORANT which is used for all experiments reported in this article. While the game is inspired by VALORANT, there are several differences: for instance, our version features a reduced playing area to accommodate the 2v2 setup (compared to VALORANT's standard 5v5), and the terrain includes a grayed-out area that is not used in gameplay. These adjustments were made to simplify the environment while preserving key tactical elements. Fig. 1(a) shows a screenshot of *Lyra:Ascent*. Each match consists of four players paired into two teams: defenders and attackers. The attackers' goal is to successfully plant and detonate a bomb within the bomb site; Fig. 1(b) shows a player attempting to plant the bomb. The defenders' goal is to defend the bomb site, winning if they successfully defuse a planted bomb or prevent the bomb from being planted. Furthermore, attackers win if all defenders are killed, and defenders win if all attackers are killed before the



(a) (b)

Fig. 2. *Lyra:Ascent* map used for our experiments. The map is heavily inspired by the Ascent level of *VALORANT* (Riot Games, 2020). (a) View of the level created in *Lyra:Ascent*. (b) Map schematic of the level where A represents the attacker spawn point, D represents the defender spawn point, and B is the bomb site.

(a) (b)

Fig. 3. (a) 15 horizontal and (b) 15 vertical rays that are distributed nonuniformly, forming a 15×15 grid-based sensory input.

bomb is planted. Here, we visualize a demo[1] of the *Lyra:Ascent* bots.

Lyra:Ascent is a tactical shooter, and as such its main mechanic is tactical navigation around the map to key locations, and eliminating enemies by shooting or using abilities. Characters in tactical shooter games often come with a large variety of abilities. To focus our study, we opt to adopt only two representative character roles. Specifically, all players on *Lyra:Ascent* are equipped with the same weapon (a pistol), and are given one of two character roles: the *Controller* or the *Initiator*. The *Controller*, on the one hand, aims to assist the *Initiator* by decreasing the enemies' access to a target area. He/she may achieve that either by throwing an incendiary grenade [as seen in Fig. 1(c)] or by launching a visual blocker in the space. The *Initiator*, on the other hand, acts more aggressively as it is equipped with skills that may stop enemies from using their own abilities or completely visually impairedthem, thereby making them easier to eliminate from the game.

The *Lyra:Ascent* game map we use in this article consists of a target bomb site and two spawning areas, as seen in Fig. 2. Each team starts in one of the two spawning areas, with the closest one to the bomb site belonging to the defending team.

IV. *Lyra:Ascent* Observation and Action Space

In imitation learning, the agent learns by observing and mimicking the behavior in expert demonstrations. This involves defining the observation and action spaces of the environment. The observation space for our agents consists of visual sensors (see Section IV-A), audio sensors (see Section IV-B), direction sensors (see Section IV-C), and game-state information (see Section IV-D), which are described in detail in the following sections. In Section IV-E, we describe the actions space.

A. Visual Sensors

In a competitive shooter game, it is crucial that game players have precise visual information that, in turn, would allow them to move and aim precisely. We employ range finders that measure the distance between the controlled character and various objects of interest via ray casting. To achieve higher degrees of precision, one usually increases the number and density of range finders. Previous research works, including neuroevolution-based approaches, such as NERO [34] and the UT^2 bot of BotPrize [22], [32], have used visual sensors, such as range finders, for agent perception. However, range finders are computationally expensive when applied in large numbers and are typically not feasible to achieve high-resolution sensory input for ML models. Range finders that are uniformly distributed in every direction result in an unnecessary amount of detail near the mid- and far-periodic vision areas, which will then require larger models for processing such detail.

To address this, we employ a nonuniform distribution of range finders, prioritizing density near the crosshair while maintaining awareness of the surroundings. Thus, inspired by human visual perception, we instead distributed a minimal number of range finders that are dense near the player's crosshair (i.e., the center of the screen) and sparse at the far peripheral (i.e., the top, bottom, corners, and both sides of the screen). As a result, a game-playing agent may need to adjust the mouse multiple times to accurately locate enemies appearing in its peripheral vision. This behavior mimics the imperfect mouse control of human players on similar occasions. Human players, however, can in contrast to our agents focus their visual perception away from the crosshair without moving the mouse.

The visual perception system of our bots casts rays across 225 directions, forming a 15×15 grid for 11 different types of game objects (see Fig. 3). Thus, our approach structures sensory data as a 3-D tensor suitable for convolutional processing, improving learning efficiency. The ten game objects that can be seen are enemies, teammates, enemy grenades, team grenades, smoke, fire, dropped bombs, planted bombs, bomb site, and finally, a layer for all remaining objects, such as walls and other static elements, in the environment's geometry. Collectively, the visual sensors form a 3-D tensor of size $15 \times 15 \times 10$ that is suitable for convolution. Our range finders are casting rays directly from the character's forehead. One of the rays is cast directly forward, while the rest are rotated along the character's pitch (vertical axis) and yaw (horizontal axis) with the following angles $[70, 45, 20, 10, 6, 3, 1, 0, -1, -3, -6, -10, -20, -45, -70]$ for yaw and $[45, 30, 20, 10, 6, 3, 1, 0, -1, -3, -6, -10, -20, -30, -45]$ for pitch. We include angles that go beyond a human player's vertical vision to ensure that our agent is aware of the terrain and other objects to avoid near its feet. If a range finder hits a game object, its distance to the agent is added to the

[1][Online]. Available: https://www.youtube.com/watch?v=sC3lMntBFHE

corresponding visual feature layer. If a ray does not hit an object, then the sensor's default value is set to the maximum distance of 100 m; all visual sensors are finally normalized within this range.

One challenge with our nonlinear distribution of range finders is that important game objects, such as grenades or enemies far away, can be missed in between two rays, especially at the mid or far peripheral areas due to their sparsity. To address this, we cast a small set of additional range finders—one for each important game object in the scene—within the character's 90° field of view, which includes all other players, grenades, fire, bombs, and the bomb site. The distance of these additional range finders are tracked and stored within the nearest cell in the corresponding feature layer.

B. Audio Sensors

Audio is an important aspect of shooters, as it can provide decisive information, such as the location of enemies and when shots are being fired. Such information can help agents make better decisions and exploit silent walking and crouching as a viable tactic during play. Our agents are equipped with eight evenly distributed directional audio sensors for each one of the following sound types: footsteps and jump sounds performed by other players, shots fired, bomb beeping, grenades exploding, and the bomb being dropped. The audio sensors are based on the sound engine of Unreal Engine 5 ensuring that game-playing agents can hear and process the same sounds that human players can. Each audio sensor captures the distance to the nearest sound type and stores this distance for each sound type that is less than 100 m away and is min–max normalized using the maximum range of 100 m. In contrast to the visual sensors, these are inverted, such that loud/nearby sounds produce a high sensory input value, and sounds far away, or no sound, produce low input.

C. Distance and Direction

The data gathering includes position variables and directions for players, teammates, bomb sites, and bombs. Specifically, *X*- and *Y*-coordinates are combined to determine the positions of various entities. Following this, Euclidean distances are calculated between the primary entity's position and the positions of other relevant entities. This process helps in deriving spatial relationships and proximities critical for subsequent analysis and modeling.

D. Game State

Game state information refers to additional information about the state of the current match and the team that is not directly collected from visual or audio sensors. Such information includes *player*-related features, such as whether the player is jumping or crouching; *bomb*-related features, such as whether the player or one of their teammates has the bomb; and finally, *team*-related features, such as the minimum distance to the nearest enemy and the time left in the current game round. Table A in the supplementary material gives details about the game state features in *Lyra:Ascent*.

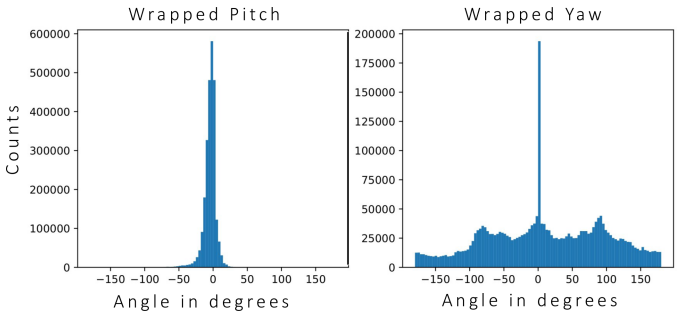


Fig. 4. Pitch and yaw actions distributions of the players.

E. Action Space

The action space has two parts: one for aiming (mouse movement) and one for all the keys that can be pressed and clicked.

1) Aiming: Instead of aiming by controlling the mouse, our agent can directly manipulate the rotation of the character's body. The action space for aiming is tightly connected to our visual sensors by offering the same set of actions equal to the angles used by the range finders. The uneven distribution of angles allows for precision when making small adjustments to the aim while also reducing the number of actions at the mid and far peripherals, resulting in a smaller action space. It potentially also makes it easier to learn a mapping between sensors and actions, as for each sensor, there is an action for that exact direction. With this setup, if the agent wants to aim toward something it sees, it has a corresponding action to do so. After investigating the distribution of actions performed by humans, some of which can be seen in Fig. 4, we decided to squeeze the top and bottom actions, resulting in the following set of angles for pitch $[20, 10, 6, 3, 1, 0, -1, -3, -6, -10, -20]$. This discrete and unevenly distributed action space is inspired by the work of Pearce and Zhu [3]. In their work, yaw and pitch are sampled independently; however, we found that this approach can lead to unintended behaviors when multiple targets are present in the agent's field of view. This issue was also extensively explored in a related study by Pearce et al. [35] that investigates human behavior imitation using diffusion models and highlighted similar challenges with independent sampling. To solve such issues, we combine the yaw and pitch angles to construct a 2-D action space with $11 \times 15 = 165$ options. Our approach directly couples sensors and actions, in the sense that there exists one action that directly corresponds to each angle used by our range finder sensors. This is a novel approach that to the best of the authors' knowledge has not been done before in this domain.

2) Key Presses: The other part of the action space is concerned with key presses and includes: W, A, S, and D (for forward, left, backward, right), Space (for jumping), 4 (bomb planting or defusal), G (dropping the bomb), R (reloading), Q (main ability), E (secondary ability), and left mouse click (shooting). The 4-key has to be held down for either 4 or 7 s to plant or defuse the bomb. Every other key will trigger an action as soon as they are pressed.

TABLE I
PRESENTED OBJECTIVES TO PLAYERS DURING DATA COLLECTION

	Attacking	Defending
Lava + Smoke	Find the best way to defend the bomb from being defused.	Catch the other team off-guard, be unexpected.
Flash + Ability Blocker	Find the best way to plant the bomb.	Make people look at you and not your teammate. Be distracting.

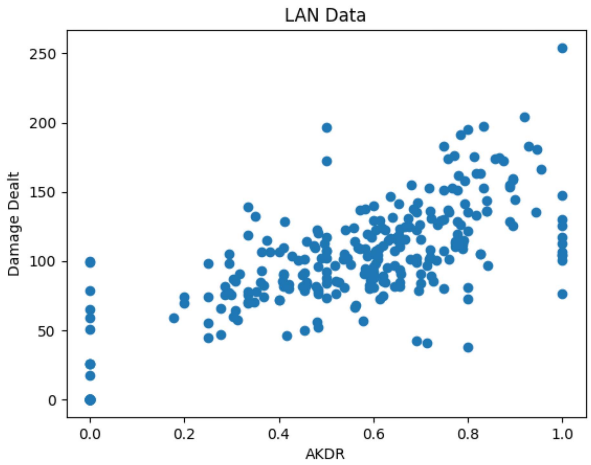


Fig. 5. Average damage dealt per round by the player.

V. DATASET

A dataset was collected from several LAN parties where different players faced off in teams playing several rounds each. We had 28 players produce a total of 48.6 h of gameplay. Prompts were given to each player according to the class they were assigned, which can be seen in Table I. The teams swapped between attacking and defending on every round but maintained the same role throughout the match. The players had a diverse set of proficiency at playing first-person tactical shooters. The kill–death ratio was calculated as $\frac{\text{number of kills}}{\text{number of deaths}}$ approximating how skilled each player is in killing enemy players. The assist–kill–death ratio (AKDR) approximates each player's contribution in killing the enemy team and was calculated as $\frac{\text{kills}+\text{assists}}{\text{kills}+\text{assists}+\text{deaths}}$. Fig. 5 shows the damage dealt versus AKDR and x-axis indicates a median AKDR at approximately 0.6. There were several teams who were never killed, indicated with AKDR of 1, with the opposing team's AKDR at 0. This explains the groupings around 0 and 1 in Fig. 5 (right). The data were then further cleaned and prepared for training. Each round was split into trajectories for each player with 16 observation–action pairs per second, as in [3], resulting in a total of 2 797 632 timesteps.

VI. NETWORK ARCHITECTURES

Imitating human behavior data exhibits complex dynamics and poses challenges for standard statistical models and ML approaches, which typically assume independent and identically distributed data. To address this, we adopt a sequential learning approach that incorporates historical information to capture the temporal peculiarities of human behavior trajectory. Memory is a critical component to successfully imitating the human play data; thus, this article presents findings using memory-based neural networks. Here, we adopt architectures with LSTM units [11] to enable memory beyond just a few frames.

We compare various sizes of the LSTM-based architecture in our experimental setup. Fig. 6 illustrates the general structure of our model. Our problem is a multioutput classification problem with a combination of binary and multiclass outputs. This type of problem is common in imitation learning, where a model learns to mimic the actions of an expert by predicting multiple outputs that represent different aspects of the human's behavior.

The model was implemented using TensorFlow Keras in Python, with inference executed in a separate process external to the game environment. Training was conducted with a learning rate of 0.0003, weight decay of 0.001, a batch size of 96, and over 600 epochs using sequences of 64 timesteps. Although we do not report detailed system resource metrics, such as memory or CPU utilization, the trained models are sufficiently lightweight to support concurrent inference for four bots on a standard laptop, demonstrating the practical efficiency of the approach. In our case, we utilize a gaming PC with an Intel Core i7-10700KF CPU at 3.80 GHz and 32 GB of RAM.

As illustrated in Fig. 6, the encoding layer of the network consists of four parallel streams (with four different colors) for each of the set of features described in Section IV. Thus, the data coming from visual sensors are stored in a cube with size $15 \times 15 \times 10$. To capture temporal dependencies, we incorporate sequential information over 64 timesteps, allowing the model to learn patterns in human behavior over time. This temporal aspect is crucial for understanding movement dynamics and decision-making sequences, allowing the bot to predict actions based on past observations rather than treating each frame independently. For inference, we use stateful models and initialize the hidden state with zeros. The audio data consist of 56 features, stored in a vector (1×56), and it is processed as a sequence of 64 time steps, allowing the model to analyze sound-based cues and their temporal evolution, which can influence decision-making in gameplay. Similarly, game state features are 29, and directional and distance information has nine features. Each stream of data inputs is associated with an encoding layer. For the game state features, audio sensor features, and distance features, this was a simple dense layer of varying sizes with linear activations. The proposed architecture utilizes a convolutional layer to encode the raycast features with dimensions $15 \times 15 \times 10$, similar to approaches in previous studies [36], [37], where convolutions have been successfully applied to process spatial data and capture local dependencies in game environments. All the networks used a convolutional layer with kernel size (3,3), stride of 1, and rectified linear unit (ReLu) activations. Each of the different-sized architectures used a different number of learnable filters. After the initial parallel streams, the outputs of these layers are flattened and concatenated to dense layers with ReLu activation. The dense layers utilize the TimeDistributed wrapper, which is a Keras/TensorFlow-specific layer that allows a dense layer to be applied independently to each time step of a sequence input.

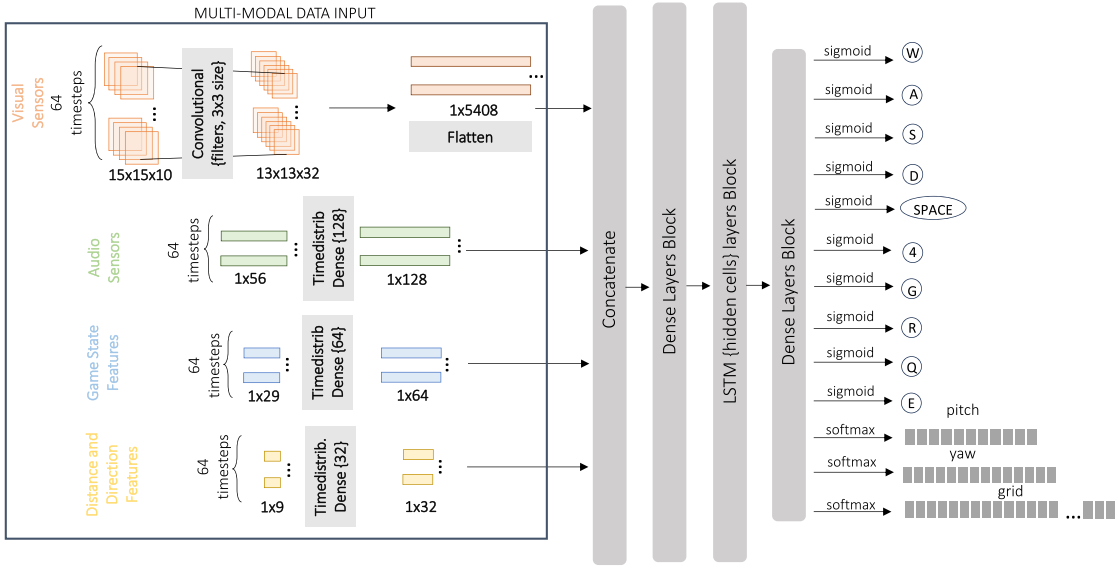


Fig. 6. Overview of the architecture and the core blocks utilized. The model integrates visual (orange), audio (green), game state (blue), and directional information (yellow). It concatenates and fuses this information into a unified representation followed by an LSTM layer to capture the temporal dimension of the data. The model performs both binary and multiclass classification, using sigmoid outputs for keypress predictions (W, A, S, D, etc.) and softmax outputs for continuous controls, such as pitch, yaw, and grid-based movements, enabling human-like gameplay behavior.

TABLE II
SIZE AND LAYOUT OF EACH MODEL

Model	Network Size	Conv. filters	Dense block	LSTM block	Dense block
A	628, 226	8	256	128	64
B	2, 348, 698	16	512	256	128
C	5, 686, 298	16	512	768	256
D	14, 881, 386	32	1024	1024	512, 256
E	25, 373, 290	32	1024, 1024	1024, 1024	1024, 512, 256
F	36, 382, 490	48	1792, 1024, 1024	1024, 1024	1024, 512, 256

Each part of the network has its own columns. Comma-separated values indicate multiple layers. Additional training parameters: learning rate: 0.0003, decay: 0.001, batch size: 96, epochs: 600, and timesteps: 64.

After the initial encoding network, the resulting features are passed to an LSTM layer with varying size. Each of the LSTM layers used dropout of 0.5. After the LSTM layer(s), the state was passed to a set of dense layers using ReLu activations. The final output of the network is then separated into several parallel streams corresponding to the different dimensions of the agent's actions, each using an appropriate activation for their loss functions. Table II gives in detail the selected hyperparameters of each model as well as its size.

Following the aforementioned architecture, this article presents networks A–F of varying size. The smallest models A–C are structurally identical besides the number of parameters in each layer. D includes two dense layers instead of one after the LSTM. E and F include two LSTM layers as well as multiple dense layers before and after the LSTM block. Each of these networks is trained with the same procedures (see Table II) and is compared in Section VIII.

VII. TRAINING METHODS

Assuming a game environment with states $x \in X$ and actions $a \in A$, the imitation learning problem is to leverage a set of expert demonstrations $U = \{(a_1, x_1), \ldots, (a_N, x_N)\}$ to find the probability distribution over possible actions that imitates the demonstrator's policy as closely as possible.

Behavior cloning uses a set of demonstrations $(a_i, x_i) \in U_i \forall i \in \{1, \ldots, N\}$ to learn a policy π that imitates the state–action mapping in U. This can be accomplished through supervised learning techniques, where the difference between the predicted action and the expert's action is minimized with respect to some metric. Concretely, the goal is to solve the optimization problem $\theta = \arg\min_{\theta} \sum_i^N L(a_i, \pi_\theta(\hat{a}_i|x_i))$ where $\pi_\theta(\hat{a_i}|x_i)$ is the probability distribution over possible actions for a given state.

To solve this optimization problem, we use backpropagation when training the models described in Section VI. In addition, since our model architecture includes LSTM layers, we use backpropagation through time or BPTT [38], and specifically the truncated version. BPTT essentially works by unrolling the recursive parts of the network and treating each time step as a state and action pair, i.e., (a_i, x_i) mentioned above.

VIII. RESULTS

To evaluate the practicality and the human-likeness of our trained models, we first investigated the inference time needed by the trained models and then conducted a two-step human evaluation process. Initially, each model controlled all four players in a 2v2 match, generating 4 h of data for both attacking and defending scenarios. For these two scenarios, we perform quantitative comparisons between the generated bot data and the human dataset used for training. In particular, we compare the distributions of different behavioral characteristics, including: speed, round length, shots per round, shots per kill, kills per round, and abilities per round. These occurrences are counted for both the attacking and defending sides. As regards the attacking scenario: bomb plant attempts and successful bomb planting

TABLE III
CPU INFERENCE TIME

Model	Parameters	Inference Time (ms)
A	628 226	1.95 ± 0.35
B	2 348 698	2.71 ± 0.50
C	5 686 298	5.16 ± 0.91
D	14 881 386	9.59 ± 0.86
E	25 373 290	16.85 ± 1.21
F	36 382 490	18.78 ± 1.47
CS:GO [3]	5 964 475	24.10 ± 3.70

attempts, and for the defending side: bomb defusal attempts and successful bomb defusal attempts. We note that all the event occurrences are per-player, not per team.

A. Inference Time

Evaluating CPU inference times allows for informed decisions regarding model selection based on performance requirements and constraints. We compare the inference time of our models to a similar model from the literature that also uses convolutional and LSTM layers that was trained for *CS:GO* [3]. We use a medium-range desktop gaming PC with an Intel Core i7-10700KF CPU at 3.80 GHz processor and 32 GB RAM and run inference on the CPU. Table III gives the size of the models we trained and the inference time of those models on the above hardware. All of our models significantly outperform the *CS:GO* model in terms of inference time. Notice that our F model has six times as many parameters but still runs significantly faster. In addition, our C model is comparable in size but is almost five times faster. This is likely due to the high cost of running convolution on large pixel input, where our low-resolution input requires far fewer of these operations. The key insight here is that pixel frames have a uniform distribution of data across the entire frame where our sensors are dense only where precise information is important and sparse where it is less important. This difference is likely smaller when running inference on a GPU. However, in our experience, the overhead of running these relatively small models on the GPU makes it slower than running them on the CPU.

B. Distributional Similarity

We collect gameplay data for each model through a series of matches where they play against themselves. In these matches, we collect key properties of the gameplay, such as the round duration, average movement speed, shots fired, shots per kill, number of kills, bomb planting attempts (for attackers), bomb defusal attempts (for defenders), and number of ability activations. This evaluation dataset consists of 4 h of active player data per model for each side (attackers and defenders). We then compare the distributional similarities of these gameplay properties by using the *Jensen–Shannon divergence*. Jensen–Shannon (JS) divergence is a symmetric way of measuring the similarity between two probability distributions. JS divergence leverages the *Kullback–Liebler divergence* (KL divergence), which is an asymmetric measure of distribution similarity. JS divergence

TABLE IV
JENSEN–SHANNON DIVERGENCE FOR COMPARING EACH OF OUR TRAINED MODELS (A–F) AGAINST THE RECORDED HUMAN PLAYER DATA

Models	bot A	bot B	bot C	bot D	bot E	bot F
ATTACK						
Duration	0.119	0.043	0.048	0.029	**0.027**	0.041
Speed	0.654	**0.050**	0.148	0.545	0.403	0.303
Shots	0.010	0.016	0.010	**0.008**	0.024	0.012
Shots/Kills	0.020	0.007	**0.006**	0.010	0.015	0.011
Kills	0.011	0.004	0.004	**0.002**	0.003	0.002
Plt. Attempts	0.003	0.004	0.003	**0.001**	0.004	0.021
Abilities	0.050	0.004	0.004	0.013	0.011	**0.003**
DEFENSE						
Duration	0.113	0.036	0.037	**0.023**	0.023	0.035
Speed	–	**0.249**	0.511	0.616	0.558	0.276
Shots	0.016	0.016	**0.005**	0.011	0.014	0.021
Shots/kills	0.021	0.015	0.009	0.008	**0.006**	0.012
Kills	0.006	0.001	0.001	**0.001**	0.001	0.003
Def. Attempts	0.027	0.028	0.024	0.018	**0.013**	0.016
Abilities	0.057	0.014	0.009	0.014	0.011	**0.008**

The smaller divergences are highlighted in bold.

works by computing the KL divergence between the provided distributions [P and Q in (1)] and a mixture of those distributions [M in (1)], and then taking the average of those KL divergence values. Concretely, JS divergence is defined as

$$\mathrm{JS}(P,Q) = \frac{1}{2}(\mathrm{KL}(P||M) + \mathrm{KL}(Q||M))$$
$$\text{where } M(x) = \frac{1}{2}(P(x) + Q(x)). \tag{1}$$

The underlying KL divergence formula is defined as

$$\mathrm{KL}(P||Q) = \sum_{x \in X} P(x) \log \frac{P(x)}{Q(x)}. \tag{2}$$

The comparative analysis between human behavior and that of our agents provides valuable insights into the models' representation fidelity and the degree of similarity between the gameplay patterns reproduced by the models and those observed in human behavior. This analysis helps to validate and later refine the models, and also helps us understand the differences between human and model behaviors.

Table IV gives the JS divergence computed across a number of recorded gameplay and behavioral features, and comparing each of our trained models (A–F) against the recorded human player data. From Table IV, we can see that model D achieves the best JS divergence across more of the dimensions than any of the other trained models. In particular, model D performs most closely to the human data in terms of shots fired per round ("Shots"), kills, and bomb planting attempts ("Plt. Attempts") while on the attacking team, and round duration ("Duration") and kills while on the defending team. Furthermore, there is only one measure for which model D performs very poorly, and that is for the average moving speed ("Speed"). Model D is ranked worst and second worst for this feature while on the attacking and defending teams, respectively. For the other measures, where it is neither best nor worst, it tends to land middle of the pack or near the best.

Note that while model D struggles with movement speed, so do most of the other models! Model B achieves the best

TABLE V
COMPARISON OF PERFORMANCE METRICS BETWEEN HUMAN PLAYER DATA AND A–F TRAINED BOTS ANALYTICS FOR ENEMY KILL PERCENTAGE, SUCCESSFUL PLANT PERCENTAGE, AND SHOTS PER KILL DURING ATTACK MODE

	stuck	at least one enemy kill %	planted successful %	shots per kill
human	0	45%	8%	2
bot A	10	34%	11%	5
bot B	16	40%	9%	3
bot C	6	38%	10%	2
bot D	5	46%	11%	2
bot E	9	41%	10%	3
bot F	18	43%	9%	2

TABLE VI
COMPARISON OF PERFORMANCE METRICS BETWEEN HUMAN PLAYER DATA AND A–F TRAINED BOTS ANALYTICS FOR ENEMY KILL PERCENTAGE, SUCCESSFUL DEFUSE PERCENTAGE, AND SHOTS PER KILL DURING DEFENSE MODE

	stuck	at least one enemy kill %	defused successful %	shots per kill
human	0	42%	7%	2
bot A	20	34%	5%	5
bot B	16	43%	4%	3
bot C	10	46%	6%	3
bot D	5	42%	6%	2
bot E	9	44%	7%	3
bot F	16	47%	7%	5

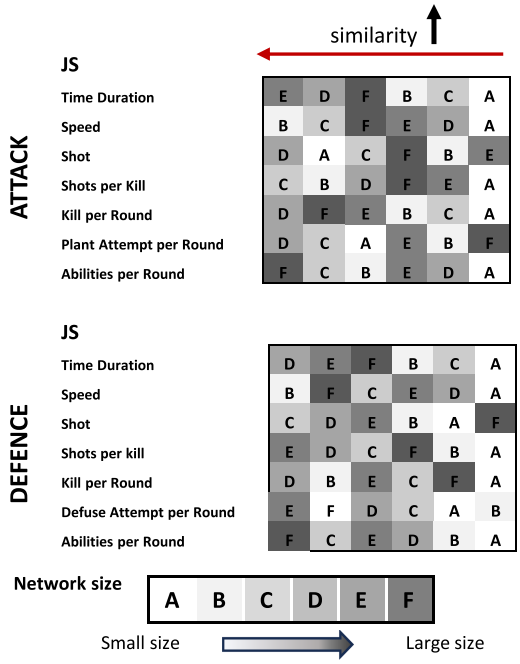


Fig. 7. Comparative results based on JS divergence between the different models. Dark colors illustrate large size models, whereas lighter colors have been selected for the smaller size models. The most similar-to-humans models on the left-hand side.

JS divergence score by a wide margin for this feature on both attacking and defending teams. If we look at Figs. 16 and 17 in the supplementary material, we can see that there is generally not much overlap for the bot and human distributions for this feature across the models. The models have a tendency to maintain a higher average movement speed throughout the rounds, shifting their distributions to the right. In fact, for model A, we can see that there is no overlap between the model and human distributions.

Another observation of note is that the larger models (E and F) struggle with replicating the shots fired distribution from the human data. E struggles more on attack, and F struggles more on defense. If we look at Fig. 14, we can see that E's shots fired distribution is skewed more heavily to the lower values, and we can see the same in Fig. 15 for F. In fact, as the model size grows, there is a general trend of the models being more conservative in their number of shots fired, resulting in tail end of the distributions (around the 40–60 buckets) being more and more sparse.

We include plots showing the distributions for some of the features in Table IV. Namely, we have plots for round duration (see Figs. 12 and 13), average movement speed (see Figs. 16 and 17), and shots fired per round (see Figs. 14 and 15). These plots show the human distribution compared to trained bot distributions while on attacking and defending teams.

Tables V and VI present a comparative analysis of performance metrics between human players and AI-trained bots (A–F) during both attack and defense modes. The metrics include the number of times a bot got stuck, the percentage of enemy kills, the percentage of successful plants, and the shots per kill. The stuck metric refers to situations where an agent remains confined to a small spatial region or performs highly repetitive actions for an extended period, without making meaningful progress toward the task objectives. This condition was identified based on qualitative observations of agent behavior, including scenarios, such as getting trapped in a corner or entering a loop of nonproductive movement. In attack mode (see Table V), Bot D closely mirrors human performance with a 46% kill rate (slightly above human 45%), a high 11% plant success rate, and a low 2 shots per kill, while getting stuck only five times. In defense mode (see Table VI), Bot D again matches human behavior with a 42% kill rate and two shots per kill, while achieving slightly lower 6% plant success and maintaining minimal path-finding problems.

Fig. 7 shows a visual ranking of the trained models based on their JS divergence scores for each of the compared features. Models located to the left are more similar to the human data compared to those located to the right for each feature. The top grid refers shows the similarities while on the attacking team, and the bottom grid is while on the defending team. From these similarity comparisons, we can conclude that model D is the best (quantitatively) performing model from those we trained. Furthermore, we observed that model D is more reactive to enemies and sustains longer fights. Model D also follows the enemies, evades shots, and uses abilities in a more targeted fashion during fights. However, these are of course just our anecdotal observations; in Section IX, we walkthrough a study we conducted to determine how realistic and believable others would find model D.

C. Evaluating Spatial Similarity in Human-Like Behavior

In order to be able to compare the moving patterns of the bots with that typical for human players, we have conducted further

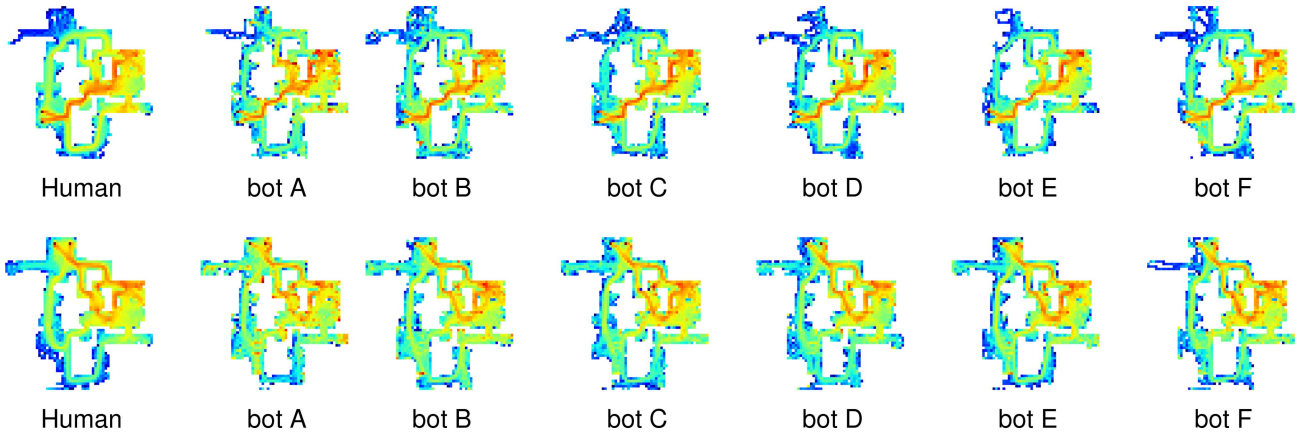


Fig. 8. (Top) Indicative heatmaps showing the map coverage of humans and bots when attacking and (bottom) defending. It is noticeable that the smallest model (bot A) gets stuck in various spots across the map.

TABLE VII
DISTANCE MEASURES COMPARING THE HEATMAP GENERATED FROM HUMAN TRAINING DATA TO A HEATMAP CREATED FOR EACH MODEL A–F

Models	bot A	bot B	bot C	bot D	bot E	bot F
			ATTACK			
EMD 1-D (no location)	4.4	**0.6**	1.8	1.4	4.4	0.8
EMD 2-D Euclidean	4.146	**1.825**	2.491	2.529	3.680	2.535
ASD	0.603	0.449	0.407	**0.364**	0.431	0.414
			DEFENSE			
EMD 1-D (no location)	8.6	6.2	2.6	**2**	3.2	2.2
EMD 2-D Euclidean	4.143	4.779	3.155	**2.689**	3.681	3.844
ASD	0.634	0.531	0.427	**0.381**	0.450	0.450

Best value in bold.

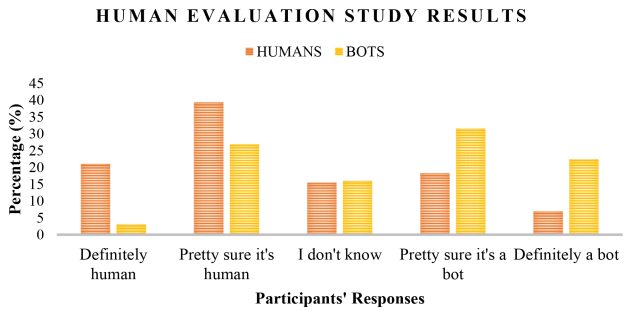


Fig. 9. Replies of the human evaluation questionnaire with 32 questions on whether the presented video clip is human or a bot player. The study involved 20 participants who evaluated the realism of the bots.

analysis using heatmaps imposed on the game map. Heatmaps in the context of an agent playing a video game visualize the spatial distribution of the agent's presence across different locations on the game map. Each location on the map is represented by a point, with the different points drawing the trajectory of the agent and the colored scale presenting the intensity of the agent's presence at each point. Cooler colors, such as blue or green, indicate areas where the player spends less time or engages in fewer actions. Warmer colors, such as yellow, orange, or red, indicate areas with high player activity or presence. These colorings can be seen in Fig. 8.

Table VII contains details about the quantitative comparison of the heatmaps visualizing player location for bots and humans. As the heatmaps and the previous section on behavioral characteristics show, different aspects of human behavior are reproduced by the various models to varying degrees. In the following, we quantify some of these differences in moving patterns by comparing the resulting heatmaps numerically. Since the heatmaps can be considered distributions of how much time was spent in which location, and inspired by previous research [3], we use the Earth mover distance (EMD) as a way to compare [see also (3) and (4)]. EMD (also called Wasserstein distance) works by quantifying the minimal work required to transform one distribution into another and offers a robust metric for comparing human and bot distributions.

Assuming 1-D EMD and let $\mathcal{J}(P, Q)$ denote the set of all joint distributions J for (x, y) that have marginals P and Q. $J(x, y)$ indicates how much "mass" must be transported from x to y in order to transform the distributions P into the distribution Q. Then, EMD is the "cost" of the optimal transport plan and is computed by

$$\text{EMD}(P, Q) = \inf_{J \in \mathcal{J}(P,Q)} \mathbb{E}_{(x,y)\sim J}[\ ||x - y||\ ]. \tag{3}$$

The $\mathcal{P}, \mathcal{Q}$ distributions are represented as histograms with buckets. The notation $||x - y||$ is equal to the absolute difference $|x - y|$, which is the cost of transporting the mass between the bucket x and the bucket y, which in the 1-D case is simply the absolute difference in their positions.

EMD is also applied using 2-D distributions, making it useful for comparing heatmaps in an interpretable manner. For 2-D, we are comparing how much time was spent in different locations on a discretized grid. In this case, the cost of transportation between any two locations $x = (x_1, x_2)$ and $y = (y_1, y_2)$ is the Euclidean distance between the corresponding cells in the grid

$$\text{EMD}(P, Q) = \inf_{J \in \mathcal{J}(P,Q)} \mathbb{E}_{(x,y)\sim J}\left[\sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}\right] \tag{4}$$

where P and Q represent the 2-D distributions over the grid. Also, (x_1, x_2) and (y_1, y_2) are the coordinates of two points in the 2-D grid. The distance $\sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$ is the

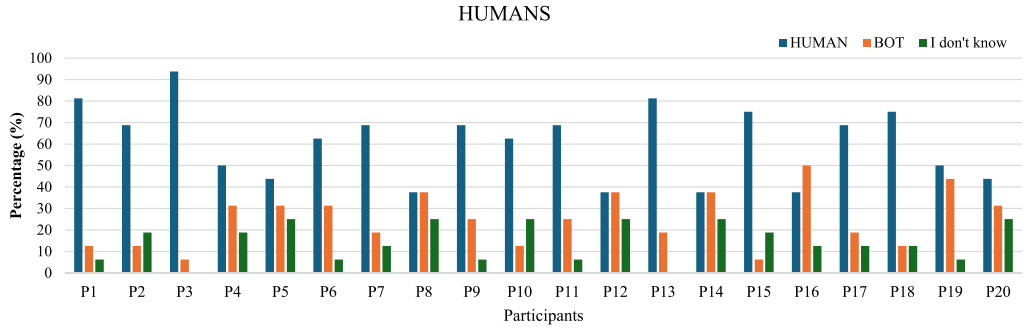


Fig. 10. Replies of the human evaluation questionnaire with 16 questions showing human players. The study shows the responses per participant for the total of 20 participants. High blue/human bars show that all raters recognized humans more often than they mistook them for bots.

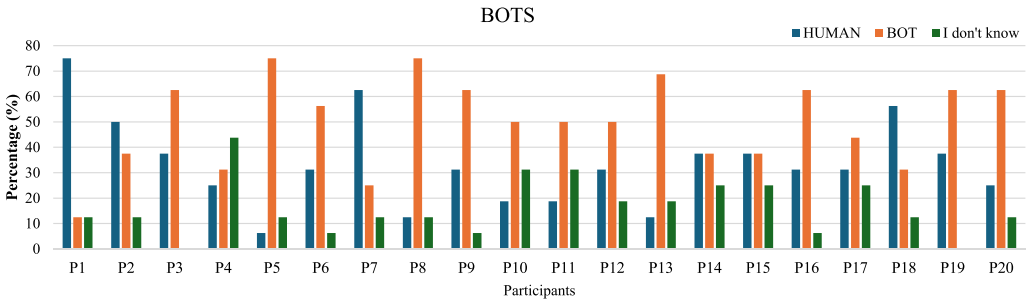


Fig. 11. Replies of the human evaluation questionnaire with the rest of 16 questions showing bot players. The study shows the responses per participant for the total of 20 participants. The high blue/human bars for P1, P2, P7, and P18 show that these raters were often fooled into thinking a bot was a human.

Euclidean distance between the two grid cells, representing the cost to move mass from one grid cell to another.

In order to add context and baselines, we compute EMD between the distributions of how much time was spent in a given cell, ignoring its locations [see (3)]. Each distribution is calculated as a histogram with buckets. This approach thus should be able to identify if one heatmap contains more instances of the player being "stuck" or "camping" than the other. It would, however, not be able to differentiate between the two. Furthermore, we compute EMD on the 2-D grid to compare how much time was spent in which location in the map, discretized as cells. We are using the Euclidean distance on the grid to determine the cost of moving between the different cells. For example, moving from cell (0,0) to (4,0) incurs a cost of 4, and so does moving to (0,4) instead [see (4)]. This approach thus compares if the same relative amount of time was spent in the same locations. This reduces the risk of counting a player being "stuck" as intentional "camping", as camping only makes sense in specific locations. The absolute sum of the difference (ASD) of matrix cells was also calculated as a baseline.

Fig. 8 visualizes how much time the player and agents spent at each location on the map. From the figure, we can see that the bots' heatmaps more closely mirror the human heatmaps while attacking than while defending. In attack scenarios, bots B–F demonstrate broad spatial coverage and high activity along the central corridors, similar to human players, as indicated by the comparable red and orange regions. In contrast, during defense, bots—particularly A and B—exhibit highly localized behavior, spending disproportionate time in areas with deep red hotspots. This leads to concentrated red regions and less diverse coverage than seen in the human heatmap. This is further reflected in Table VII by the generally lower EMD values for attacking over defending. When comparing model performance overall, it seems like there is a sweet spot in terms of model size right around where model D is. The smallest model, A, has some of the biggest distance values observed, and while the larger models, E and F, perform decently overall, they still perform considerably worse than C and D in defense mode. The only exception is the smaller model, B, which performs very well in attacking mode, but really poorly when playing defense. Based on a visual comparison of the heatmaps, the distances obtained by B in attack mode are most likely due to more varied behavior at the opposing team's spawn point and less frequent utilization of the balcony area. The latter, however, highly depends on the defending agents' behavior as this is frequently the location of fights due to proximity to the bomb site. Conversely, when model B is on the defending team, it does not behave like human players. This is reflected in the obtained distance measures and can also be seen in the heatmaps. Namely, the model B bots seem to cover the map much more evenly than human players, especially in the bottom area. Based on the quantitative comparisons of the heatmaps, model D is the most robust in replicating human-like moving patterns across game modes and different measures.

D. Qualitative Analysis of Bots' Behavior

Based on our observations of both human and bot behaviors, we note that the bots' navigation patterns closely mirror those of human players. Their micromovement behaviors effectively replicate the subtle adjustments and positioning strategies commonly seen in real gameplay. Furthermore, the bots successfully execute key game mechanics, such as planting and diffusing the bomb, and display considerable proficiency in shooting, making them formidable opponents. However, there have been observed rare cases where the bots diverge from human behavior, such as occasionally forgetting about previously seen enemies, failing to react to nearby sounds or attacks from behind, and getting briefly stuck in certain map areas. These shortcomings likely stem from sensor limitations or the need for additional training data. By addressing these issues—whether through better sensory modeling, improved reinforcement signals, or targeted dataset expansion—we aim to further refine bot behaviors to better capture the rare and nuanced actions that humans display.

IX. Human Perception of Gameplay Authenticity

This section presents the approach we followed to evaluate our methods via human feedback. We first detail the protocol we used for the purposes of comparing our trained bots against human players (see Section IX-A). Then, we go through the analysis of the data obtained and we discuss the core findings of this set of experiments (see Section IX-B).

A. Experimental Protocol

In this study, we designed a postgame evaluation using predefined videos extracted from $2v2$ matches of *Lyra: Ascent* featuring either human players or our trained bots. After the matches, we carefully selected and edited videos that showcase various in-game situations for use in a comparative questionnaire. The purpose of the questionnaire was to evaluate the human likeness of the bots by asking participants to classify each video as either human or bot, providing valuable insight into the effectiveness of the bots in mimicking human behavior in gaming scenarios.

The selection process for the videos shown to participants followed a systematic approach to minimize bias. Several matches were recorded, and the first minutes of each session were automatically divided into 15-s clips. The idea of short video clips was adopted following the studies as in [26] and [39]. The selection criteria aimed to exclude clips where the player was dead or where visual artifacts, such as map loading, were present, as these elements would not have provided the viewer with information regarding the behavior of the bots. This selection process resulted in 16 videos featuring bots and 16 featuring human players, for a total of 32 videos. The human gameplay clips used in the evaluation were selected to represent a diverse cross section of gameplay styles and skill levels. All bot gameplay clips used in the study were exclusively selected from Model D, as it demonstrated the most human-like behavior in our analysis. The videos were presented in a randomized order to 20 participants, all of whom were experienced gamers. Crucially, videos were not chosen based on specific behavioral displays to avoid selection bias. Instead, a quasi-random approach was employed, ensuring that the selected clips provided insight into navigation, combat behavior, and other behaviors relevant to the gameplay without explicitly selecting for them. This methodology was applied consistently to both human and bot gameplay videos, ensuring a fair and unbiased selection process.

Prior to participating in the study, all participants were provided with a detailed information sheet outlining the purpose, procedures, and potential risks associated with the experiment. This document ensured that participants were fully aware of their rights and the nature of the tasks they would be engaging in. Participants were informed that their participation was voluntary and that they could withdraw at any time without any consequences.

B. Data Analysis and Findings

The human evaluation results provide promising insights into the realism of the bots, demonstrating that they are effective at mimicking human behavior and appearance. From the total of 20 participants * 32 videos $=$ 640 ratings, the half of them—320—are human ratings. As regards human ratings, $39\% + 21\% = 60\%$ correctly identified as human videos, while $18\% + 7\% = 25\%$ mistakenly labeled as bots and 15% expressed uncertainty, as shown in Fig. 9. As regards bot ratings, $27\% + 3\% = 30\%$ of responses suggested that bot videos were actually human, which highlight the bots' ability to display traits typically associated with human behavior convincingly. Interestingly, this humanness rating aligns closely with the results of the UT^2 BotPrize competition, where the built-in Unreal Tournament 2004 bot achieved a similar rating [22]. While $32\% + 22\% = 54\%$ of participants correctly identified the bots, the fact that nearly a third mistook them for humans points to a level of sophistication in the bots' behavior that can deceive viewers. In addition, only 16% of participants were unsure about the nature of the bot videos, which is comparable to the 15% uncertainty rate for human videos, further emphasizing that the bots are achieving a similar level of ambiguity as genuine human videos and that the behaviors presented were not imbalanced in terms of clarity or other extraneous factors for either humans or bots.

In the below analysis, we illustrate the responses per participant (P1–P20). Based on human evaluation results, our gameplay bots exhibit a high degree of behavioral realism. Although participants were generally able to identify human players with high accuracy (see Fig. 10), their ability to distinguish bot behavior was significantly lower, as shown in Fig. 11, often resulting in misclassification or uncertainty. The frequent "I don't know" responses suggest that the bots' actions and decisions during gameplay closely resemble those of human players. This indicates that bots successfully emulate human-like strategies and interactions, making them difficult to distinguish from actual human participants.

Milani et al. [26] in their paper evaluated three AI agents—symbolic, hybrid, and reward-shaping—against human players. The symbolic and hybrid agents were correctly identified as nonhuman 83% of the time, while the reward-shaping agent achieved a median accuracy of 50%, meaning participants could not reliably distinguish it from human players. This result suggests that the reward-shaping agent effectively mimicked human navigation patterns, passing the human navigation Turing test. In comparison, our study found that 30% of the participants misclassified the behavior of the bot as human, indicating a moderate level of humanlikeness in our bots. Although our results demonstrate a convincing level of realism, there is still room for improvement to reach the indistinguishability achieved by the most human-like agent in [26].

X. Conclusion

This article presents a practical approach for building efficient, human-like ML-based bots for multiplayer first-person shooter games, aiming to bridge academic methods with industry needs. Our imitation learning framework leverages deep neural networks with temporal structure and compute-efficient sensors to operate effectively within real-time constraints of commercial game environments, merely focusing in the aspects referred to below.

1) *Data collection:* We constructed a dataset of human gameplay trajectories by capturing player actions and deploying a novel set of sensors that record spatial and audio context. This sensor design balances computational cost and responsiveness, supporting real-time inference.

2) *Human-like ML models:* Using supervised learning in temporal data, our models replicate human behavior patterns to enhance immersion. Our most effective policy model contains approximately 15 million parameters with an average inference time of 9.59 ms per decision on a consumer-grade gaming PC, highlighting its viability for deployment.

3) *Multifaceted evaluation of bot believability:* We assessed the bots' realism through distributional comparisons, spatial heatmaps, and human feedback. These evaluations consistently favored a model (Model D) using convolutional and LSTM layers, which achieved strong alignment with human behavior while maintaining computational efficiency.

A. Future Work

Human behavior is characterized by stochasticity and multimodality and exhibits structured correlations across action dimensions. Advancements in imitation learning algorithms beyond behavior cloning have witnessed a notable convergence with techniques, such as reinforcement learning (RL) or behavioral transformers, fostering more adaptive and robust learning frameworks to compensate with human behavior in unseen scenarios. Applying RL to our dataset is an interesting area of research as it has the potential to create diverse behaviors by actively exploring the environment and recovering the optimal reward and agent policy. Using algorithms, such as generative adversarial imitation learning, to improve the robustness and diversity of learned policies [40] can lead to more robust and naturalistic policies. Recently, approaches, such as the decision Transformer (DT) [41] and the trajectory Transformer [42], were introduced to serve as alternatives for offline RL algorithms. The generalized DT, is another variant of the extended transformer models family that can be applied for our dataset [43], [44].

Future work could explore interactive evaluation settings, such as humans playing against bots or measuring outcomes in AI-vs-AI and AI-vs-idle matches, to better understand agent behavior in adversarial and passive scenarios. Investigating whether humans consistently act in alignment with the coupled sensory-action inputs, or whether more variable, imperfect responses are typical, could offer valuable insights into the necessity of direct coupling. Ablation studies that perturb or offset the action space (for example, by a few degrees) may clarify the impact of the coupling on performance. In addition, analyzing how well the model differentiates between roles (e.g., controller versus initiator) and whether it supports distinct cooperative behaviors remains an important future direction, especially given the importance of role dynamics in both human strategy and team-based gameplay.

References

[1] D. Silver et al., "A general reinforcement learning algorithm that masters chess, Shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.
[2] G. N. Yannakakis and J. Togelius, *Artificial Intelligence and Games*, vol. 2. Berlin, Germany: Springer, 2018.
[3] T. Pearce and J. Zhu, "Counter-strike deathmatch with large-scale behavioural cloning," in *Proc. IEEE Conf. Game*, 2022, pp. 104–111.
[4] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
[5] P. Hingston, *Believable Bots*. Berlin, Germany: Springer, 2012.
[6] O. Vinyals et al., "Starcraft II: A new challenge for reinforcement learning," 2017, *arXiv:1708.04782*.
[7] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, "Imitation learning: A survey of learning methods," *ACM Comput. Surv.*, vol. 50, no. 2, pp. 1–35, 2017.
[8] B. Zheng, S. Verma, J. Zhou, I. W. Tsang, and F. Chen, "Imitation learning: Progress, taxonomies and challenges," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 5, pp. 6322–6337, May 2022.
[9] P. Ladosz, L. Weng, M. Kim, and H. Oh, "Exploration in deep reinforcement learning: A survey," *Inf. Fusion*, vol. 85, pp. 1–22, 2022.
[10] Z. Liu et al., "Tail: Task-specific adapters for imitation learning with large pretrained models," 2023, *arXiv:2310.05905*.
[11] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
[12] N. M. Shafiullah, Z. Cui, A. A Altanzaya, and L. Pinto, "Behavior transformers: Cloning k modes with one stone," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 22955–22968.
[13] S. Dasari and A. Gupta, "Transformers for one-shot visual imitation," in *Proc. Conf. Robot Learn.*, 2021, pp. 2071–2084.
[14] S. Reed et al., "A generalist agent," 2022, *arXiv:2205.06175*.
[15] A. R. Farhang, B. Mulcahy, D. Holden, I. Matthews, and Y. Yue, "Human-like behavior in a third-person shooter with imitation learning," in *Proc. IEEE Conf. Games*, 2024, pp. 1–4.
[16] V. Sreeramdass, R. R. Paleja, L. Chen, S. van Waveren, and M. Gombolay, "Generalized behavior learning from diverse demonstrations," in *Proc. 1st Workshop Out-of- Distrib. Generalization Robot.*, 2023, pp. 1–13.
[17] Y. Jiang, J. Z. Kolter, and R. Raileanu, "Uncertainty-driven exploration for generalization in reinforcement learning," in *Proc. Deep Reinforcement Learn. Workshop NeurIPS*, 2022, pp. 1–32.
[18] V. Mnih et al., "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
[19] C. Berner et al., "Dota 2 with large scale deep reinforcement learning," 2019, *arXiv:1912.06680*.

[20] P. R. Wurman et al., "Outracing champion Gran Turismo drivers with deep reinforcement learning," *Nature*, vol. 602, no. 7896, pp. 223–228, 2022.
[21] C. Zhang et al., "Training interactive agent in large FPS game map with rule-enhanced reinforcement learning," in *Proc. IEEE Conf. Games*, 2024, pp. 1–8.
[22] J. Schrum, I. V Karpov, and R. Miikkulainen, "Human-like combat behaviour via multiobjective neuroevolution," in *Believable Bots: Can Computers Play Like People?* Berlin, Germany: Springer, 2012, pp. 119–150.
[23] A. Shih, S. Ermon, and D. Sadigh, "Conditional imitation learning for multi-agent games," in *Proc. 17th ACM/IEEE Int. Conf. Hum.-Robot Interaction*, 2022, pp. 166–175.
[24] J. Harmer et al., "Imitation learning with concurrent actions in 3D games," in *Proc. IEEE Conf. Comput. Intell. Games*, 2018, pp. 1–8.
[25] M. Jaderberg et al., "Human-level performance in 3D multiplayer games with population-based reinforcement learning," *Science*, vol. 364, no. 6443, pp. 859–865, 2019.
[26] S. Milani et al., "Navigates like me: Understanding how people evaluate human-like AI in video games," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2023, pp. 1–18.
[27] J. Togelius, G. N. Yannakakis, S. Karakovskiy, and N. Shaker, "Assessing believability," in *Believable Bots: Can Computers Play Like People?* Berlin, Germany: Springer, 2012, pp. 215–230.
[28] P. Hingston, "The 2 k botprize," in *Proc. IEEE Symp. Comput. Intell. Games*, 2009, pp. 1–1.
[29] N. Shaker et al., "The turing test track of the 2012 Mario AI championship: Entries and evaluation," in *Proc. IEEE Conf. Comput. Inteligence Games*, 2013, pp. 1–8.
[30] J. Ortega, N. Shaker, J. Togelius, and G. N Yannakakis, "Imitating human playing styles in super Mario bros," *Entertainment Comput.*, vol. 4, no. 2, pp. 93–104, 2013.
[31] E. Camilleri, G. N Yannakakis, and A. Dingli, "Platformer level design for player believability," in *Proc. IEEE Conf. Comput. Intell. Games*, 2016, pp. 1–8.
[32] J. Schrum, I. V. Karpov, and R. Miikkulainen, "UT 2: Human-like behavior via neuroevolution of combat behavior and replay of human traces," in *Proc. IEEE Conf. Comput. Intell. Games*, 2011, pp. 329–336.
[33] E. Zuniga et al., "How humans perceive human-like behavior in video game navigation," in *Proc. CHI Conf. Hum. Factors Comput. Syst. Extended Abstr.*, 2022, pp. 1–11.
[34] K. O. Stanley, B. D. Bryant, and R. Miikkulainen, "Real-time neuroevolution in the NERO video game," *IEEE Trans. Evol. Comput.*, vol. 9, no. 6, pp. 653–668, Dec. 2005.
[35] T. Pearce et al., "Imitating human behaviour with diffusion models," 2023, *arXiv:2301.10677*.
[36] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
[37] G. Lample and D. S. Chaplot, "Playing FPS games with deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, vol. 31, pp. 1–7.
[38] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proc. IEEE*, vol. 78, no. 10, pp. 1550–1560, Oct. 1990.
[39] D. Durst et al., "Learning to move like professional counter-strike players," *Comput. Graph. Forum*, vol. 43, no. 8, 2024, Art. no. e15173.
[40] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 1–14.
[41] L. Chen et al., "Decision transformer: Reinforcement learning via sequence modeling," in *Proc. 35th Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 15084–15097.
[42] M. Janner, Q. Li, and S. Levine, "Offline reinforcement learning as one big sequence modeling problem," in *Proc. 35th Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 1273–1286.
[43] H. Furuta, Y. Matsuo, and S. S. Gu, "Generalized decision transformer for offline hindsight information matching," 2021, *arXiv:2111.10364*.
[44] S. Sudhakaran and S. Risi, "Skill decision transformer," 2023, *arXiv:2301.13573*.